

El flujo de información en la red, paso a paso

Qué entra y qué sale en cada etapa, con las formas de los tensores
(datos FEM $\rightarrow$ grafos $\rightarrow$ MPNN $\rightarrow$ GENERIC $\rightarrow \Delta T \rightarrow loss$)

Resumen

Este documento traza, **como con lápiz y papel**, el camino que recorre la información dentro de nuestra arquitectura: desde una simulación FEM hasta la pérdida que entrena la red. En cada etapa decimos exactamente **qué entra, qué se hace y qué sale**, anotando la **forma (shape)** de cada tensor (en **rojo**). Usamos un **ejemplo de juguete** (una malla de $N=4$ nodos) para que las formas sean concretas. No se asume nada de los otros documentos; aquí solo importa “¿por dónde va el dato y con qué forma?”.

Índice

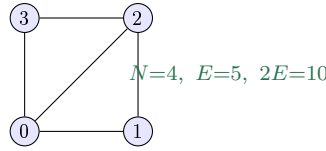
1. Notación y ejemplo de juguete	1
2. Vista de pájaro: el camino completo	2
3. Etapa 0 — Los datos: una simulación FEM	2
4. Etapa 1 — De la simulación FEM a grafos	3
5. Etapa 2 — Normalización	4
6. Etapa 3 — El MPNN (Encoder + Processor)	4
6.1. Encoder: subir a la latente	4
6.2. Processor: $L=8$ pasos de mensajes	4
7. Etapa 4 — El head GENERIC entálpico	5
8. Etapa 5 — De ΔT a la temperatura siguiente (rollout)	6
9. Etapa 6 — Cómo se construye la pérdida (push-forward K)	7
10. Etapa 7 — Validación y selección del modelo	7
11. Resumen: qué entra y qué sale en cada etapa	8

1. Notación y ejemplo de juguete

Símbolos que usaremos para las **dimensiones**:

Símbolo	Significado	En el ejemplo de juguete
N	nº de nodos de la malla	$N = 4$
E	nº de aristas no dirigidas	$E = 5$
$2E$	aristas dirigidas (ambos sentidos)	$2E = 10$
S	nº de fotogramas temporales de la sim	p.ej. $S = 350$
d	ancho de la latente (hidden_dim)	$d = 128$
L	nº de bloques del Processor	$L = 8$
K	pasos del push-forward (multi-step)	$K = 2$

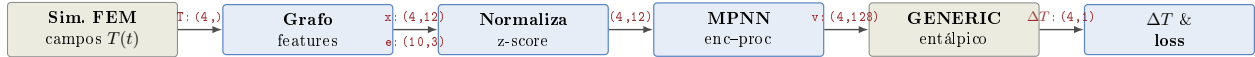
Ejemplo de juguete. Una placa cuadrada partida en 2 triángulos: $N=4$ nodos, $E=5$ aristas (4 lados + 1 diagonal), $2E=10$ dirigidas. Mantendremos estos números entre corchetes para que veas las formas *con números*. En la realidad N es de cientos a miles y varía por simulación.



En cristiano: Cada “shape” (**filas**, **columnas**) dice cuántos números hay. $(4, 12) = 4$ nodos, 12 features por nodo. Si te pierdes, mira siempre la forma: te dice qué representa el tensor.

2. Vista de pájaro: el camino completo

Antes del detalle, el mapa entero con las formas en cada flecha (ejemplo de juguete, **un** par de fotogramas):



Las siguientes secciones abren cada caja. La regla de oro: **el número de filas casi siempre es N** (una fila por nodo) o $2E$ (una por arista); lo que cambia es el número de *columnas* (de 12 features crudas, a 128 latentes, a 1 que es ΔT).

3. Etapa 0 — Los datos: una simulación FEM

El dato de partida es una **simulación FEM** ya resuelta (un objeto `SimulationResult` guardado en `.npz`). Contiene la película térmica y todo lo necesario para reconstruir las features:

Campo	Forma	Qué es
coords	$(N, 2)$	posición (x, y) de cada nodo
cells	$(M, 3)$	los 3 nodos de cada triángulo
times	$(S,)$	instante de cada fotograma
temperature	(S, N)	temperatura de cada nodo en cada instante
source_position	$(S, 2)$	posición de la antorcha en cada instante
source_power	$(S,)$	potencia del arco (0 cuando se apaga)
metadata	dict	Goldak, contornos, geometría/trayectoria

Entra: nada (es el punto de partida): una placa, unos parámetros de proceso.
Hace: el solver FEM resuelve la ecuación del calor instante a instante.
Sale: `temperature (S, N)`: la película de temperaturas (la “verdad”).

En cristiano: Una simulación = una película. `temperature[t]` es la foto del instante t : la temperatura de los N nodos. Hay S fotos. De aquí saldrán $S-1$ ejemplos de entrenamiento (un par de fotos consecutivas cada uno).

4. Etapa 1 — De la simulación FEM a grafos

Cada par de fotogramas consecutivos $(t, t+1)$ se convierte en un grafo. Una simulación de S fotos da $S-1$ grafos. Cada grafo lleva cuatro cosas:

Tensor del grafo	Forma	Qué es
<code>x</code>	$(N, 12)$	12 features por nodo, estado en t
<code>edge_index</code>	$(2, 2E)$	qué nodo conecta con qué nodo (emisor, receptor)
<code>edge_attr</code>	$(2E, 3)$	3 features por arista dirigida
<code>y</code>	$(N, 1)$	objetivo: $\Delta T = T^{t+1} - T^t$

Las 12 features de nodo (orden fijo). Solo información *relativa*: nada de coordenadas absolutas.

#	Nombre	Qué es
0	<code>T</code>	temperatura actual del nodo (la única que se realimenta)
1	<code>q_goldak</code>	fuerza del arco evaluada en el nodo (0 al enfriar)
2,3	<code>dx, dy_local</code>	posición del nodo relativa a la antorcha (marco tangente/normal)
4	<code>net_power</code>	potencia neta ηP (constante por sim)
5	<code>speed</code>	velocidad de soldadura
6,7,8	<code>bc_*</code>	one-hot del tipo de nodo (interior/Dirichlet/Robin)
9	<code>h_conv</code>	coef. de convección local
10	<code>emissivity</code>	emisividad local
11	<code>T_inf</code>	temperatura ambiente local

Las 3 features de arista: $[\Delta x, \Delta y, \|\mathbf{u}_{ij}\|]$ con $\mathbf{u}_{ij} = \mathbf{x}_i - \mathbf{x}_j$ (el desplazamiento entre los dos nodos). La geometría entra *solo* por aquí.

Aristas bidireccionales. Cada arista de la malla se mete en **los dos sentidos**: por eso `edge_index` tiene $2E$ columnas. La fila 0 son los emisores i , la fila 1 los receptores j .

Entra: una `SimulationResult` (la película FEM).

Hace: para cada t : monta las 12 features por nodo, las 3 por arista, y la resta $T^{t+1} - T^t$.

Sale: una lista de $S-1$ grafos, cada uno con `x` $(N, 12)$, `edge_index` $(2, 2E)$, `edge_attr` $(2E, 3)$, y `y` $(N, 1)$.

Idea clave 4.1 (El objetivo es ΔT , no T). La red **no** predice la temperatura nueva directamente: predice el **incremento** $\Delta T = T^{t+1} - T^t$ $(N, 1)$. Luego se suma: $T^{t+1} = T^t + \Delta T$. Predecir el cambio (pequeño) es más fácil y estable que predecir el valor absoluto.

En cristiano: Un grafo = una foto (N nodos $\times$ 12 números) + la respuesta correcta (ΔT por nodo). Las aristas dicen quién es vecino de quién y a qué distancia. Eso es *todo* lo que ve la red.

5. Etapa 2 — Normalización

Las redes entrenan mejor si las entradas tienen media 0 y desviación 1. Se calcula la media y la desviación de cada columna *sobre todo el corpus* (se guardan en `stats.pt`) y se aplica un **z-score**:

$$x_{\text{norm}}[:, c] = \frac{x[:, c] - \mu_c}{\sigma_c}.$$

Detalles que importan:

- Las **columnas one-hot** (6,7,8) *no* se tocan (ya son 0/1).
- Las **edge_attr** se pasan **crudas** (sin normalizar).
- El objetivo y también se normaliza para la pérdida; al predecir se des-normaliza con `inverse_y`.

Entra: `x` (**N,12**) crudas (kelvin, etc.).

Hace: resta la media y divide por la desviación, columna a columna (menos los one-hot).

Sale: `x_norm` (**N,12**) (mismas dimensiones, ahora adimensional).

En cristiano: La forma **no cambia** (**N,12**): solo cambian los *valores* (pasan a “cuántas desviaciones por encima de la media”). Las constantes μ, σ se guardan con el modelo, así que en inferencia se aplica la misma transformación.

6. Etapa 3 — El MPNN (Encoder + Processor)

El MeshGraphNet tiene tres partes; aquí usamos Encoder y Processor (el Decoder se *omite* en nuestro modelo, lo sustituye el head GENERIC, §7).

6.1. Encoder: subir a la latente

Dos MLPs independientes llevan las features crudas al espacio latente de ancho $d=128$:

$$\underbrace{\mathbf{x_norm}}_{(N,12)} \xrightarrow{\text{MLP nodo}} \underbrace{\mathbf{v}}_{(N,128)}, \quad \underbrace{\mathbf{edge_attr}}_{(2E,3)} \xrightarrow{\text{MLP arista}} \underbrace{\boldsymbol{\epsilon}}_{(2E,128)}.$$

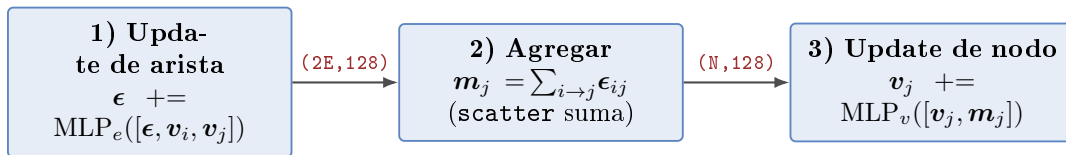
Entra: `x_norm` (**N,12**) y `edge_attr` (**2E,3**).

Hace: cada MLP expande sus features a 128 dimensiones.

Sale: latentes de nodo $\mathbf{v}$ (**N,128**) y de arista $\boldsymbol{\epsilon}$ (**2E,128**).

6.2. Processor: $L=8$ pasos de mensajes

Es el corazón. Se repiten $L=8$ **bloques**. En cada bloque pasan 3 cosas (con conexiones *residuales*: se suma la entrada):



Las formas dentro de un bloque:

- Update de arista: la entrada del MLP es $[\epsilon_{ij}, v_i, v_j]$, o sea $(2E, 384) (= 128+128+128) \rightarrow (2E, 128)$.
- Agregación: **scatter** suma, en cada nodo receptor, los mensajes de sus aristas: $(2E, 128) \rightarrow (N, 128)$. *Aquí* es donde la información salta de un nodo a sus vecinos.
- Update de nodo: la entrada del MLP es $[v_j, m_j]$, $(N, 256) \rightarrow (N, 128)$.

Tras $L=8$ bloques, la información de cada nodo ha viajado hasta **8 aristas** de distancia (el “radio de visión” alrededor del arco).

Entra: $v (N, 128)$, $\epsilon (2E, 128)$, $\text{edge_index} (2, 2E)$.

Hace: 8 rondas de: actualizar aristas, sumar mensajes en los receptores, actualizar nodos.

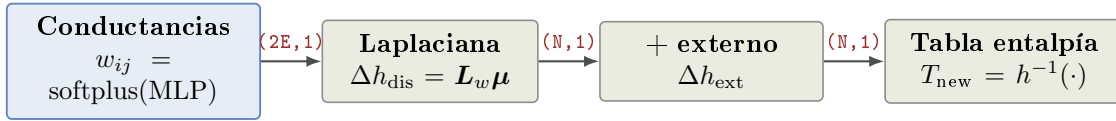
Sale: **latentes de nodo** $v (N, 128)$ (ricas en contexto local). Es lo que consume el head GENERIC.

Idea clave 6.1 (El número que importa al salir del MPNN). Sale $v (N, 128)$: **un vector de 128 números por nodo**. No es todavía una temperatura ni un ΔT ; es un “resumen aprendido” del estado de cada nodo y su entorno. La etapa siguiente lo convierte en física.

En cristiano: **scatter** = “por cada nodo, suma lo que le llega de sus aristas”. Repetido 8 veces, mezcla la información del vecindario. Entra $(N, 12)$, sale $(N, 128)$: más columnas = más contexto por nodo, todavía sin unidades físicas.

7. Etapa 4 — El head GENERIC entálpico

Aquí las latentes se convierten en un ΔT que **respeto la física**. El head recibe tres cosas: las latentes $v (N, 128)$, la conectividad $\text{edge_index} (2, 2E)$, y las features crudas $x (N, 12)$ (de donde saca la **temperatura física** T , la fuente q , el ambiente T_∞).



Paso a paso, con formas:

1. **Temperatura física.** Des-normaliza la columna 0: $T (N, 1)$ en kelvin. La fuerza termodinámica es $\mu = 1/T (N, 1)$.
2. **Conductancias aprendidas.** Por cada arista, un MLP recibe features *simétricas* de sus dos extremos $[v_i + v_j, (v_i - v_j)^2, T_i + T_j, |T_i - T_j|] (2E, 258)$ y saca un escalar no negativo $w_{ij} = w_{ji} \geq 0 (2E, 1)$. (Esta es la *única* parte aprendida del head.)
3. **Disipación (Laplaciana).** Por cada arista $i \rightarrow j$ se aporta $w_{ij}(\mu_j - \mu_i)$ al receptor y se suma con **scatter**: $\Delta h_{\text{dis}} (N, 1)$. Su suma por grafo es **cero** $\Rightarrow$ conserva energía (1.er principio). El calor va de caliente a frío por construcción.
4. **Término externo.** Aporte del arco (Goldak) y enfriamiento por los bordes: $\Delta h_{\text{ext}} (N, 1)$.
5. **Recuperar la temperatura por la tabla de entalpía.** Dos interpolaciones contra la curva tabulada $H(T)$ (que lleva el calor latente):

$$H_{\text{cur}} = H(T) (N, 1), \quad T_{\text{new}} = H^{-1}(H_{\text{cur}} + \Delta h_{\text{dis}} + \Delta h_{\text{ext}}) (N, 1),$$

$$\Delta T = T_{\text{new}} - T (N, 1).$$

6. **Normalizar la salida.** $\Delta T_{\text{norm}} = (\Delta T - \mu_y)/\sigma_y$ (**N,1**), para cumplir el mismo contrato de salida que el modelo base.

Entra: latentes $\mathbf{v}$ (**N,128**), `edge_index` (**2,2E**), features crudas $\mathbf{x}$ (**N,12**).

Hace: conductancias $\rightarrow$ Laplaciana (conserva energía) + externo $\rightarrow$ inversión de la curva de entalpía.

Sale: ΔT_{norm} (**N,1**) (incremento de temperatura, normalizado).

Idea clave 7.1 (Dónde está lo aprendido y dónde la física). De todo el head, **lo único aprendido** son las conductancias w_{ij} (un MLP pequeño sobre las latentes). El resto — $\mu = 1/T$, la suma Laplaciana, la tabla $H(T)$ y la inversión h^{-1} — es **física fija, sin parámetros**. Por eso la conservación de energía y el segundo principio se cumplen *siempre*, no “si entrena bien”.

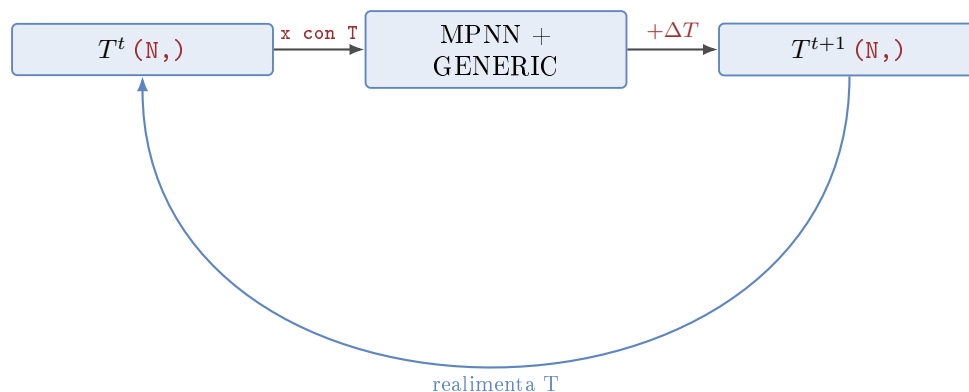
En cristiano: Entra un vector de 128 por nodo; sale **un** número por nodo: el ΔT . Por el camino, la red solo decide “cómo de bien conduce cada arista” (w_{ij}); el resto es termodinámica de libro que garantiza que no se invente ni pierda energía.

8. Etapa 5 — De ΔT a la temperatura siguiente (rollout)

La salida ΔT_{norm} se des-normaliza a kelvin y se suma:

$$T^{t+1} = T^t + \underbrace{\text{inverse_y}(\Delta T_{\text{norm}})}_{(\mathbf{N},) \text{ en kelvin}}.$$

En **inferencia** esto se encadena: el modelo arranca de T^0 y predice toda la película, **realimentando su propia predicción**. Clave: *solo* se realimenta la columna de temperatura; las otras 11 features (fuente, coordenadas co-móviles, etc.) se conocen de antemano para todos los pasos (la trayectoria es determinista).



Entra: T^0 (**N,**) y la trayectoria conocida.

Hace: repite ~ 1500 veces: sobrescribe la columna T, pasa por la red, suma ΔT .

Sale: la película predicha $\hat{T}$ (**S,N**), comparable con la verdad FEM.

En cristiano: Inferencia = “dame la primera foto y te doy toda la película”. Cada ΔT se suma y la nueva T vuelve a entrar. Solo la temperatura se realimenta; todo lo demás (dónde está el arco, etc.) ya se sabe.

9. Etapa 6 — Cómo se construye la pérdida (push-forward K)

En **entrenamiento** no basta con un paso: entrenamos con **ventanas de K pasos** (push-forward) para alinear el objetivo con el rollout largo. El *dataloader* sirve, por cada ejemplo, una **ventana** de K grafos consecutivos de la misma simulación: $[g_t, g_{t+1}, \dots, g_{t+K-1}]$.

El bucle de una ventana (con $K=2$):

1. **Sembrar:** $\hat{T} \leftarrow T^t$ verdadero + ruido **(N,)**.
2. **Para cada paso $k = 0, \dots, K-1$:**
 - (a) poner $\hat{T}$ en la columna de temperatura del grafo g_{t+k} ;
 - (b) normalizar, pasar por la red $\rightarrow \Delta T$, des-normalizar **(N,)**;
 - (c) predicción: $T_{\text{pred}} = \hat{T} + \Delta T$ **(N,)**;
 - (d) verdad: $T_{\text{true}} = T^{t+k} + y_{t+k}$ **(N,)** (la T real del paso);
 - (e) acumular pérdida: $\mathcal{L} += \frac{1}{N} \sum_i \left(\frac{T_{\text{pred},i} - T_{\text{true},i}}{\sigma_T} \right)^2$;
 - (f) **realimentar:** $\hat{T} \leftarrow T_{\text{pred}}$ *sin detach*.
3. **Promediar:** $\mathcal{L} \leftarrow \mathcal{L}/K$; **backward**, recortar gradiente, **optimizer.step**.

$$\mathcal{L} = \frac{1}{K} \sum_{k=0}^{K-1} \frac{1}{N} \sum_{i=1}^N \left(\frac{T_{\text{pred},i}^{(k)} - T_{\text{true},i}^{(k)}}{\sigma_T} \right)^2$$

(un MSE en temperatura física, escalado por la desviación σ_T).

Entra: una ventana de K grafos consecutivos (en crudo, físicos).

Hace: desenrolla K pasos realimentando la predicción; mide el error contra la verdad de cada paso.

Sale: un escalar $\mathcal{L}$ del que se hace **backward** a través de *todos* los K pasos (BPTT).

Idea clave 9.1 (La línea que define el push-forward). El $\hat{T} \leftarrow T_{\text{pred}}$ **sin detach** es lo que hace que el gradiente atravesase los K pasos. Si pusieras **detach**, cada paso entrenaría aislado (objetivo de un paso) y no se alinearía con el rollout largo. $K=1$ es el caso clásico; $K=2$ ya fuerza al operador a ser contractivo en dos pasos.

En cristiano: Entrena “jugando a predecir 2 pasos seguidos con tu propia predicción de por medio”. El error de los 2 pasos se suma y se propaga hacia atrás de una vez. Así la red aprende a no acumular errores, que es justo lo que falla en los rollouts largos.

10. Etapa 7 — Validación y selección del modelo

La métrica que decide “qué modelo guardamos” **no** es la pérdida de un paso: es el **RMSE de un rollout autoregresivo completo** (§8) sobre las simulaciones de validación (que el modelo no vio en entrenamiento, por el *split por simulación*). Se guarda el *checkpoint* que minimiza ese RMSE.

Entra: el modelo actual + las sims de validación.

Hace: rueda la película entera de cada sim y compara con la verdad FEM.

Sale: un RMSE medio (kelvin). Si mejora, se guarda **best_model.pt**.

En cristiano: Validar = “despliega la película completa en datos nuevos y mira cuánto te desvías”. Es más exigente que mirar un solo paso, y es lo que de verdad nos importa para usar el modelo.

11. Resumen: qué entra y qué sale en cada etapa

Etapa	Entra (forma)	Sale (forma)
0. FEM	parámetros	<code>temperature (S,N)</code> (la verdad)
1. Grafos	1 <code>SimulationResult</code>	$S-1$ grafos: <code>x (N,12)</code> , <code>edge_index (2,2E)</code> , <code>edge_attr (2E,3)</code> , y <code>y (N,1)</code>
2. Normalización	<code>x (N,12)</code>	<code>x_norm (N,12)</code> (misma forma)
3. Encoder	<code>(N,12)</code> y <code>(2E,3)</code>	<code>v (N,128)</code> , <code>ϵ (2E,128)</code>
3. Processor ($\times 8$)	<code>v (N,128)</code> , <code>ϵ (2E,128)</code>	<code>v (N,128)</code> (contexto local)
4. GENERIC head	<code>v (N,128)</code> , <code>edge_index</code> , <code>x (N,12)</code>	<code>ΔT_{norm} (N,1)</code>
5. Rollout	<code>T^t (N,)</code>	$T^{t+1} = T^t + \Delta T (N,)$
6. Pérdida (K)	ventana de K grafos	escalar $\mathcal{L}$ (BPTT por K pasos)
7. Validación	modelo + sims val	RMSE de rollout (selección)

Idea clave 11.1 (La regla mnemotécnica del flujo). **Filas** = “por nodo” (N) o “por arista” ($2E$); **columnas** = “cuánta información”. El viaje es: $12 \rightarrow 128$ (el MPNN enriquece) $\rightarrow 1$ (el head GENERIC destila a un ΔT físico). Y ese ΔT se suma a T y vuelve a entrar: una y otra vez en inferencia, K veces con gradiente en entrenamiento.